

Partie 2 : Préparation des Données

Compléments de Mathématiques 2 – Introduction au Machine Learning

HAMLIL Mohamed PARDO TERAN German
EL KORAICHI Mohamed Yassine ANZID Keltoum

1736895600000

Table des matières

1 Chargement des données explorées	1
2 Feature Engineering	1
3 Construction du jeu de données final	2
4 Traitement des valeurs manquantes	3
5 Séparation train / test	3
6 Sauvegarde	4

1 Chargement des données explorées

```
data Rated <- readRDS(here::here("output", "data Rated.rds"))  
cat("Données chargées :", nrow(data Rated), "lignes", ncol(data Rated), "colonnes\n")
```

2 Feature Engineering

Nous transformons les variables textuelles (accords, notes) en indicatrices binaires pour les accords/notes les plus fréquents. La fonction `extract_top_features()` définie dans `R/Utils.R` est utilisée pour cette tâche.

```

# Extraire les top accords
result_accords <- extract_top_features(data_rated, "Main_Accords", top_n = 10, prefix = "accords_")
data_rated <- result_accords$data
cat("Top accords :", paste(result_accords$items, collapse = ", "), "\n")

# Extraire les top notes de tête
result_top <- extract_top_features(data_rated, "Top_Notes", top_n = 8, prefix = "topnote_")
data_rated <- result_top$data
cat("Top notes de tête :", paste(result_top$items, collapse = ", "), "\n")

# Extraire les top notes de cœur
result_mid <- extract_top_features(data_rated, "Middle_Notes", top_n = 8, prefix = "midnote_")
data_rated <- result_mid$data

# Extraire les top notes de fond
result_base <- extract_top_features(data_rated, "Base_Notes", top_n = 8, prefix = "basenote_")
data_rated <- result_base$data

```

3 Construction du jeu de données final

```

# Nombre de notes pour chaque parfum (mesure de complexité)
data_rated <- data_rated %>%
  mutate(
    nb_top_notes = str_count(Top_Notes, ",") + ifelse(!is.na(Top_Notes), 1, 0),
    nb_mid_notes = str_count(Middle_Notes, ",") + ifelse(!is.na(Middle_Notes), 1, 0),
    nb_base_notes = str_count(Base_Notes, ",") + ifelse(!is.na(Base_Notes), 1, 0),
    nb_accords = str_count(Main_Accords, ",") + ifelse(!is.na(Main_Accords), 1, 0)
  )

# Encoder Concentration
data_rated <- data_rated %>%
  mutate(Concentration = ifelse(is.na(Concentration), "Unknown", Concentration))

# Garder les concentrations fréquentes, regrouper les autres
top_concentrations <- data_rated %>%
  count(Concentration, sort = TRUE) %>%
  head(8) %>%
  pull(Concentration)

```

```

data_rated <- data_rated %>%
  mutate(Concentration = ifelse(Concentration %in% top_concentrations,
                                Concentration, "Autre"))
data_rated$Concentration <- factor(data_rated$Concentration)

# Sélectionner les variables pour la modélisation
feature_cols <- c(
  "Release_Year", "Rating_Count", "Concentration",
  "nb_top_notes", "nb_mid_notes", "nb_base_notes", "nb_accords",
  grep("^accord_|^topnote_|^midnote_|^basenote_", names(data_rated), value = TRUE)
)

df_model <- data_rated %>%
  select(all_of(c(feature_cols, "Satisfaction")))

cat("Variables du modèle :", ncol(df_model) - 1, "\n")
cat("Observations :", nrow(df_model), "\n")

```

4 Traitement des valeurs manquantes

! Choix de conception

L'imputation par la **médiane** est privilégiée par rapport à la moyenne car elle est robuste aux valeurs extrêmes présentes notamment dans `Rating_Count`.

```

preprocess_median <- preProcess(df_model[, feature_cols], method = "medianImpute")
df_model[, feature_cols] <- predict(preprocess_median, df_model[, feature_cols])

cat("Valeurs manquantes restantes :", sum(is.na(df_model)), "\n")

```

5 Séparation train / test

Nous utilisons une répartition **70% / 30%** avec stratification sur la variable cible.

```

set.seed(42)
train_index <- createDataPartition(df_model$Satisfaction, p = 0.7, list = FALSE)
train_data <- df_model[train_index, ]
test_data <- df_model[-train_index, ]

```

```
cat("Train :", nrow(train_data), "observations\n")
cat("Test :", nrow(test_data), "observations\n")
cat("Proportion Oui (train) :", round(mean(train_data$Satisfaction == "Oui"), 3), "\n")
cat("Proportion Oui (test) :", round(mean(test_data$Satisfaction == "Oui"), 3), "\n")
```

6 Sauvegarde

```
saveRDS(train_data, here::here("output", "train_data.rds"))
saveRDS(test_data, here::here("output", "test_data.rds"))
cat("Données sauvegardées dans output/\n")
```